



Universidad Don Bosco
FACULTAD DE INGENIERÍA
ESCUELA DE COMPUTACIÓN
DESARROLLO DE SOFTWARE EMPRESARIAL
Unidad III: Prototipado de producto



Desafío 3 (opción 2)

Sistema de Gestión Empresarial con .NET

ENUNCIADO DEL EXAMEN

Si usted ha sido contratado como desarrollador .NET para implementar un sistema de gestión de clientes y órdenes para una empresa mediana. Se solicita crear un sistema, debe ser seguro, eficiente y proveer reportes empresariales.

Tiempo: 48 horas para completar todas las funcionalidades.

OBJETIVOS DE APRENDIZAJE

Al finalizar este examen, el estudiante demostrará capacidad para:

- Implementar sistemas seguros con autenticación y autorización
- Optimizar rendimiento con sistemas de caché
- Generar reportes empresariales profesionales
- Garantizar calidad mediante pruebas unitarias
- Integrar múltiples tecnologías en una solución cohesiva

RÚBRICA DE EVALUACIÓN

Componente	Puntaje	Criterios de Evaluación
Funcionalidad Core	25%	Endpoints funcionan correctamente, CRUD completo
Seguridad & Identity	20%	Autenticación, autorización, roles implementados
Optimización (Redis)	15%	Caché implementado correctamente en endpoints clave
Reportes (SSRS)	15%	3 reportes creados, publicados y accesibles
Calidad (Pruebas)	15%	Pruebas unitarias implementadas y pasando

Componente	Puntaje	Criterios de Evaluación
Estructura & Documentación	10%	Código organizado, README claro, instrucciones

REQUERIMIENTOS TÉCNICOS

1. CONFIGURACIÓN INICIAL

- Crear solución [ASP.NET](#) Core 8 Web API
- Configurar Entity Framework Core con SQL Server
- Implementar patrón Repository o similar
- Manejo adecuado de dependencias en Program.cs

2. MODELADO DE DATOS

Base de datos: CompanyManagement

Tablas requeridas:

- **Clientes** (Id, Nombre, Email, FechaRegistro)
- **Órdenes** (Id, ClienteId, FechaOrden, MontoTotal)

Relaciones:

- Un cliente puede tener múltiples órdenes
- Una orden pertenece a un solo cliente

3. AUTENTICACIÓN Y AUTORIZACIÓN

- Implementar [ASP.NET](#) Core Identity
- Crear sistema de registro e inicio de sesión
- Implementar roles: "User" y "Admin"
- Proteger endpoints según roles
- Manejo de tokens o cookies de sesión

4. ENDPOINTS REQUERIDOS

Endpoints Públicos:

- POST /api/auth/register - Registro de usuarios
- POST /api/auth/login - Inicio de sesión

Endpoints Protegidos (Requieren autenticación):

- GET /api/clientes - Listar todos los clientes
- GET /api/clientes/{id} - Obtener cliente específico
- POST /api/clientes - Crear nuevo cliente
- PUT /api/clientes/{id} - Actualizar cliente
- DELETE /api/clientes/{id} - Eliminar cliente
- GET /api/ordenes - Listar todas las órdenes
- GET /api/ordenes/{id} - Obtener orden específica
- POST /api/ordenes - Crear nueva orden
- GET /api/ordenes/cliente/{clienteId} - Órdenes por cliente

Endpoints Admin (Requieren rol Admin):

- GET /api/reportes/clientes-activos - URL reporte 1 SSRS
- GET /api/reportes/ingresos-clientes - URL reporte 2 SSRS
- GET /api/reportes/clientes-inactivos - URL reporte 3 SSRS

5. IMPLEMENTACIÓN DE REDIS

- Configurar conexión con Redis
- Implementar caché en:
 - Listado de clientes
 - Obtención de cliente por ID
 - Listado de órdenes
- Invalidar caché al crear, actualizar o eliminar
- Tiempo de expiración: 10 minutos

6. REPORTES SSRS

Crear los siguientes reportes paginados:

Reporte 1: Clientes Activos

- Lista de clientes con número total de órdenes
- Solo clientes con al menos 1 orden
- Columnas: Nombre, Email, TotalÓrdenes

Reporte 2: Ingresos por Cliente

- Total de ingresos generados por cada cliente
- Columnas: Nombre, Email, MontoTotal

Reporte 3: Clientes Inactivos

- Clientes registrados en el último mes sin órdenes

- Columnas: Nombre, Email, FechaRegistro

7. PRUEBAS UNITARIAS

- Proyecto xUnit separado
- Pruebas para al menos 5 escenarios diferentes
- Usar base de datos en memoria
- Cubrir casos de éxito y error

TECNOLOGÍAS A UTILIZAR

- **Backend:** [ASP.NET](#) Core 8, Entity Framework Core
- **Base de datos:** SQL Server
- **Caché:** Redis
- **Reportes:** SQL Server Reporting Services
- **Autenticación:** [ASP.NET](#) Core Identity
- **Pruebas:** xUnit, Moq (opcional)
- **Herramientas:** Visual Studio, SSMS, Report Builder

ENTREGABLES

1. CÓDIGO FUENTE

- Solución Visual Studio completa
- Scripts SQL de creación de base de datos
- Migraciones de Entity Framework

2. REPORTE

- Archivos .rdl de los 3 reportes
- Instrucciones de publicación en SSRS

3. DOCUMENTACIÓN

- [README.md](#) con:
 - Instrucciones de instalación y configuración
 - Descripción de endpoints
 - Configuración de conexiones
 - Ejemplos de uso

4. EVIDENCIAS

- Capturas de pantalla de:
 - Postman probando endpoints con autenticación
 - Pruebas unitarias ejecutándose exitosamente
 - Reportes funcionando en navegador web
 - Redis almacenando datos (redis-cli o interfaz web)

INSTRUCCIONES ESPECÍFICAS

Para la Autenticación:

- Usar Identity con Entity Framework
- Implementar registro con email y password
- Los nuevos usuarios se asignan automáticamente al rol "User"
- Solo usuarios Admin pueden acceder a los reportes

Para Redis:

- Cachear respuestas de endpoints GET principales
- Invalidar cache en operaciones POST, PUT, DELETE
- Manejar serialización JSON correctamente

Para SSRS:

- Diseñar reportes profesionales con headers y formatos adecuados
- Publicar en servidor de reportes local o en la nube
- Integrar URLs en la API protegidas por autorización

Para Pruebas Unitarias:

- Probar al menos 2 endpoints del controlador de clientes
- Probar 2 endpoints del controlador de órdenes
- Probar 1 escenario de autenticación/autorización
- Usar Arrange-Act-Assert pattern



CRITERIOS DE DESCALIFICACIÓN

- Código plagiado de otros estudiantes
- Entrega fuera del tiempo límite
- Funcionalidades principales no implementadas
- No compila o errores críticos de ejecución



RECOMENDACIONES

1. **Planifique su tiempo:** Divida el trabajo en bloques de 4-6 horas
2. **Versionado:** Use Git para control de versiones
3. **Pruebas tempranas:** Pruebe cada componente conforme lo desarrolla
4. **Documentación:** Documente mientras desarrolla, no al final
5. **Backups:** Guarde copias de seguridad regularmente



EVALUACIÓN FINAL

La evaluación considerará:

- **Funcionalidad completa** del sistema
- **Calidad del código** y mejores prácticas
- **Cumplimiento** de todos los requerimientos
- **Creatividad** en la implementación
- **Documentación** clara y completa y video demo

¡MUCHA SUERTE! 🚀